When AES Throughput Depends on the CPU Execution Path: Evidence from x86-64 and Arm64

Marko Mihajlović[1]

Abstract: Efficient AES encryption is important for secure storage and communication, yet processor support alone does not determine the throughput available to software. This paper examines how library implementation and measurement scope influence CPU encryption performance. We analyze previously collected, validated AES measurements from Windows x86-64 and Oracle Cloud Arm64, with separate CBC and GCM experiments addressing key length and authenticated processing. At 32 MiB, the default OpenSSL CTR implementation reaches 4.03 GB/s on Arm64, compared with 0.21–0.22 GB/s for the evaluated alternative implementations. On Windows, its throughput advantage over PyCryptodome decreases from approximately 1.9 to 1.2 when timing expands from encryption processing to the instrumented benchmark operation. These findings show why fast cipher execution does not directly predict performance at a broader software interface. They provide a basis for selecting representative benchmarks and identifying where further optimization may benefit encryption services.

Keywords: AES; CPU instruction acceleration; OpenSSL EVP; PyCryptodome; Arm64; authenticated encryption; benchmarking.

1 Introduction

The Advanced Encryption Standard (AES) is a central component of systems that protect stored and transmitted data [1]. Its computational cost matters in applications ranging from bulk storage encryption to services processing short messages. In these settings, encryption shares processor time and memory bandwidth with other application functions. Understanding its performance therefore requires attention both to the cipher implementation and to the software through which an application invokes it.

Modern processors provide dedicated cryptographic instructions, while libraries select among implementations according to processor capabilities and software configuration. However, applications also initialize cipher contexts, allocate output buffers, convert data representations and transfer control between programming languages. A measurement taken within a native encryption routine excludes some of this work. A measurement around a higher-level call includes more of it. Both are useful, but each describes a different part of the computation.

[1] Singidunum University, Belgrade, Serbia; E-mail: marko.mihajlovic.23@singimail.rs

This distinction matters when evaluating native libraries through Python interfaces. Libraries implementing the same AES mode may perform different preparation and data-handling operations. Their measured difference can consequently reflect cipher implementation, interface design and timer placement. These contributions must be distinguished before using a benchmark to select a library or estimate application capacity.

This paper addresses the following research question: *how do the selected AES implementation and the scope of the timed operation affect measured CPU throughput, and which conclusions remain useful for application design?* We investigate this question through a retrospective analysis of validated measurements from a Windows x86-64 laptop and an Oracle Cloud Arm64 instance. AES-128 in counter (CTR) mode provides the main workload. Separate Windows experiments with cipher block chaining (CBC) and Galois/Counter Mode (GCM) extend the analysis to key length and authenticated processing [2, 3].

The paper makes three contributions. First, it quantifies the differences between the evaluated OpenSSL and PyCryptodome implementations on each host, including an OpenSSL capability-mask experiment on Arm64. Second, it shows how the relative performance observed on Windows changes when timing expands from encryption processing to the complete instrumented benchmark call. Third, it relates these observations to the effects of key length and message size in separate CBC and GCM workloads. Together, these contributions identify the information needed to interpret an encryption throughput figure and assess its relevance to a software service.

The analysis uses existing experimental records; no additional measurements were collected for this paper. Comparisons are made within each host and workload, with the timing differences stated explicitly. Section 2 places the study in the context of AES implementation and benchmarking research. Section 3 describes the experimental setup and estimators. Section 4 presents the results and their interpretation, followed by broader implications, limitations and future work in Section 5. Section 6 concludes the paper.

2 Related Work and Research Gap

2.1 AES implementations and workload characteristics

AES uses a fixed block size and supports 128-, 192- and 256-bit keys, with 10, 12 and 14 rounds, respectively [1]. The mode of operation determines how these block transformations are combined. CBC encryption introduces dependencies between successive blocks, whereas CTR generates keystream from independent counter values [2]. GCM combines counter-mode encryption with authentication

and includes tag generation or verification [3]. These differences affect the available parallelism and the work performed per message, so performance must be interpreted for a specified mode and operation.

Research on efficient AES implementations has examined both instruction support and software organization. Käsper and Schwabe [4] presented a bitsliced implementation for 64-bit Intel processors and investigated AES-GCM performance together with resistance to timing attacks. Their work demonstrates that implementation choices matter for both efficiency and security. Intel's AES-NI instructions provide dedicated operations for AES processing [5]. Such mechanisms explain why the same standardized cipher can have substantially different execution costs across implementations, even before application overhead is considered.

Library interfaces add another layer. OpenSSL EVP exposes cipher operations through an abstraction whose selected implementation depends on the library configuration and available processor features [6]. PyCryptodome provides AES modes through a Python interface backed by native code [7]. Neither the common algorithm name nor the availability of processor instructions guarantees that two interfaces select equivalent code or include the same work in a timed call. The present comparison therefore concerns the particular library configurations and interfaces recorded in the experiments.

2.2 Benchmark interpretation and the scope of this study

Systems research has established that apparently minor experimental choices can affect measured performance. Mytkowicz et al. [8] demonstrated measurement bias arising from experimental setup. Kalibera and Jones [9] developed a methodology that accounts for variation at different levels of repetition. These studies motivate documenting the execution environment and distinguishing repeated timings within a session from replication across sessions or machines.

For encryption benchmarks, measurement scope introduces a further interpretive issue. Timing the cipher transformation answers a question about processing prepared data. Timing a surrounding software operation also captures some combination of preparation, allocation, interface overhead and result handling. In authenticated encryption, tag processing and associated data introduce additional work. A throughput comparison is therefore meaningful only when these inclusions are identified alongside the payload size and implementation.

The practical question addressed here is how strongly implementation and timing choices alter the interpretation of CPU encryption benchmarks. This study connects AES implementation research with benchmarking methodology through documented comparisons of library paths, timing scopes and cryptographic workloads. Its contribution is empirical clarification of these relationships on two systems, rather than a new AES implementation.

3 Methodology and Experimental Setup

3.1 Platforms, datasets and correctness checks

The primary dataset contains CPU library measurements from two environments. The Windows system used an Intel Core i5-10300H with four physical cores and eight logical processors, Python 3.12.10, PyCryptodome 3.23.0 and verified 64-bit OpenSSL 3.6.5 libraries. The Oracle Cloud system used a Neoverse-N1-class Arm64 processor with two available virtual CPUs, Python 3.12.3 and OpenSSL 3.0.13. The original records preserve platform metadata, processor capabilities and source hashes. The hosts differ in operating system, library version and hardware configuration, so each is analyzed separately.

The primary campaigns evaluated AES-128 at payload sizes of 1 KiB, 16 KiB, 1 MiB, 16 MiB and 32 MiB. CTR is the principal workload presented here. Electronic codebook (ECB) measurements in the source records served as implementation diagnostics. Each configuration was measured in five sessions with seeds 2003–2007, using five warm-up repetitions and 20 measured repetitions per session. Known-answer and differential checks established ciphertext correctness before performance results were accepted. Raw observations, metadata and session summaries were retained.

Two earlier Windows campaigns provide complementary evidence. The CBC experiment compared software and AES-NI-enabled implementations for 128-, 192- and 256-bit keys; the present paper reports its 10 MiB encryption results. The GCM experiment evaluated 128- and 256-bit keys at 1 KiB, 16 KiB, 1 MiB, 16 MiB and 100 MiB. Both used five seeded sessions, three warm-up repetitions and 20 measured repetitions. These datasets retain their original timing definitions and are analyzed separately from the CTR campaign.

The GCM workload used a 96-bit nonce, a 128-bit authentication tag and 16 bytes of associated authenticated data (AAD), with no nonce reuse for distinct messages under the same key within a session. Validation included known-answer checks and rejection of modified ciphertext, tags and AAD. These checks establish functional correctness for the tested cases; they do not constitute a side-channel security evaluation.

3.2 Implementation paths and timing scopes

The CTR comparison uses PyCryptodome and native OpenSSL EVP. Both transform measurements use prepared output buffers. For OpenSSL, timestamps inside the compiled wrapper delimit the EVP Update/Final processing. For PyCryptodome, the transform interval includes Python dispatch and the library's internal foreign-function interface. Thus, the recorded transform rates describe each implementation at its measured interface; their ratio does not isolate the

relative cost of AES instructions.

A second interval, termed the *instrumented operation*, measures the Python call to the benchmark's measurement routine. It includes input checks, allocation, buffer bindings, setup, transformation, cleanup and output conversion, together with diagnostic calls and timing bookkeeping where applicable. This is the interval named `operation_envelope` in the records. It provides a broader view of the same benchmark execution, but is not an uninstrumented application request or a complete storage or network operation. The Windows dataset provides both intervals for the library comparison.

On Arm64, OpenSSL was also evaluated with `OPENSSL_armcap=0`. This setting disables multiple optimized capability paths. Comparing it with the default configuration tests sensitivity to capability selection within OpenSSL, but cannot isolate the effect of the AES extension from other disabled features. The analysis uses the archived Arm64 transform summaries and does not infer unrecorded instruction execution from capability flags.

The CBC results compare the software and AES-NI paths under the original campaign's common timing convention. GCM encryption includes AAD processing, payload encryption, output allocation and tag generation; decryption includes authentication-tag verification. Construction of a fresh GCM cipher object is excluded. Throughput counts payload bytes only, even though AAD and tag processing contribute to elapsed time. These rates therefore describe authenticated payload processing after object construction, rather than complete message latency.

3.3 Aggregation and interpretation

For a payload of B bytes processed in t nanoseconds, decimal throughput is

$$P = \frac{10^3 B}{t} \quad [\mathrm{MB/s}]. \tag{1}$$

KiB and MiB denote binary payload sizes; MB/s and GB/s denote decimal rates. Each primary result is the median of five session-level throughput medians. Session ranges, where reported, describe the smallest and largest of those five medians rather than confidence intervals.

For a reference implementation A and a candidate B measured under a common timing convention, the speedup of B relative to A is

$$S_{B/A}^{(j)} = \frac{\widetilde{t}_A^{(j)}}{\widetilde{t}_B^{(j)}}, \qquad S_{B/A} = \mathrm{median}_{j=1}^{5}\, S_{B/A}^{(j)}. \tag{2}$$

Here, $\widetilde{t}$ denotes the median elapsed time in session j; a value above one indicates faster execution by B. The CBC acceleration factors use this estimator. Approximate

CTR throughput ratios discussed below are ratios of the reported aggregate rates, and are distinguished from session-based speedups. Rounding and the order of aggregation can make these estimators differ.

The session is the unit of replication. Repetitions are not pooled to imply additional independent experiments, and observations are not excluded because they show unfavorable performance. Five sessions characterize repeatability on the measured host, but do not establish population-level behavior across processors. Comparisons across hosts or cipher modes are descriptive and are not treated as paired trials.

4 Experimental Results and Interpretation

4.1 Effect of the library implementation

Table 1 summarizes AES-128-CTR throughput at 32 MiB. On Windows, the OpenSSL transform rate is 5537 MB/s, compared with 2877 MB/s for PyCryptodome. On Arm64, default OpenSSL reaches 4030 MB/s, whereas the masked OpenSSL configuration and PyCryptodome achieve 218 and 209 MB/s, respectively. These values are medians across five sessions for each configuration.

Table 1
AES-128-CTR, 32 MiB: throughput by platform, implementation and timing scope (decimal MB/s).

Host	Implementation	Transform	Full call
Windows i5-10300H	OpenSSL EVP	5537	1479
Windows i5-10300H	PyCryptodome	2877	1202
Oracle Arm64	OpenSSL default	4030	—
Oracle Arm64	OpenSSL armcap=0	218	—
Oracle Arm64	PyCryptodome	209	—

Transform and full-call rates use the intervals defined in Section 3. Full call denotes the instrumented operation. Dashes indicate results outside the Arm64 transform dataset analyzed here.

The Arm64 difference is large and repeatable within the recorded sessions. The default OpenSSL session medians range from 4018 to 4035 MB/s, while the masked configuration ranges from 217.4 to 218.4 MB/s and PyCryptodome from 208.8 to 209.5 MB/s. The approximately 19-fold throughput ratio between default OpenSSL and PyCryptodome therefore reflects a persistent difference between the evaluated paths. The reduction under capability masking also shows that OpenSSL's optimized capability selection is important on this host. Because the mask disables several features, the result cannot be attributed exclusively to AES instructions.

Figure 1 places the 32 MiB results in the context of message size. Larger payloads sustain substantially higher OpenSSL throughput than the alternative paths on each host. At 1 KiB, the native transform interval is very short, making the calculated rate more sensitive to timer resolution and call overhead. Large-buffer throughput should consequently not be used to predict the latency of short requests. The two panels describe separate software environments; differences between them are not estimates of an intrinsic x86-64 versus Arm64 performance ratio.

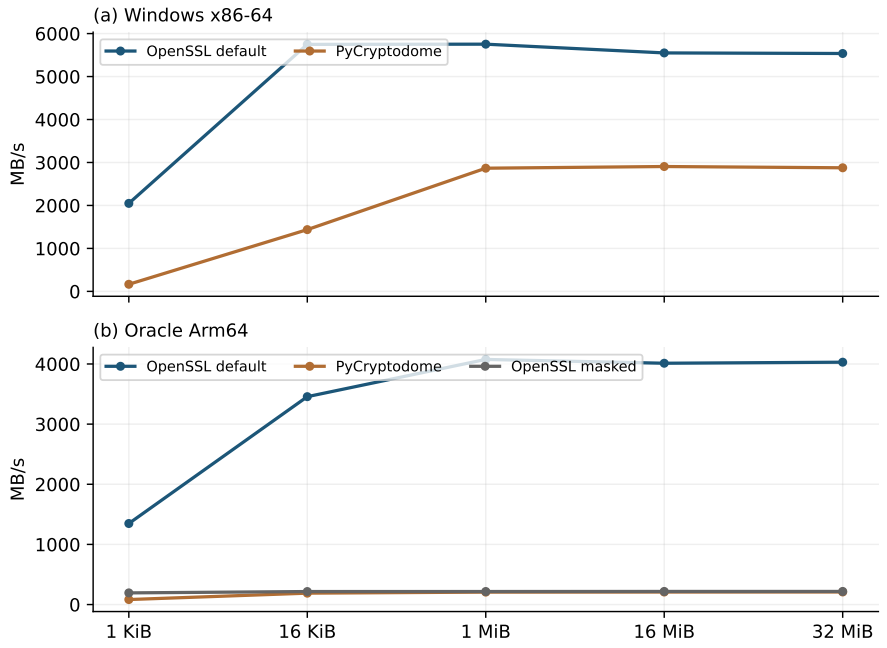


Fig. 1 — AES-128-CTR transform throughput versus payload size on (a) Windows x86-64 and (b) Oracle Arm64. Each panel compares implementations within one host.

4.2 Effect of the timed software operation

Expanding the Windows timer from transformation to the instrumented operation changes the magnitude of the library difference (Fig. 2). OpenSSL throughput decreases from 5537 to 1479 MB/s, while PyCryptodome decreases from 2877 to 1202 MB/s. The ratio of their aggregate rates consequently falls from approximately 1.93 to 1.23. Across sessions, the broader OpenSSL rate ranges from 1452 to 1625 MB/s and the PyCryptodome rate from 1179 to 1263 MB/s.

The ordering remains the same, but the advantage is smaller when additional work is timed. Selecting the faster cipher path therefore does not produce an equally

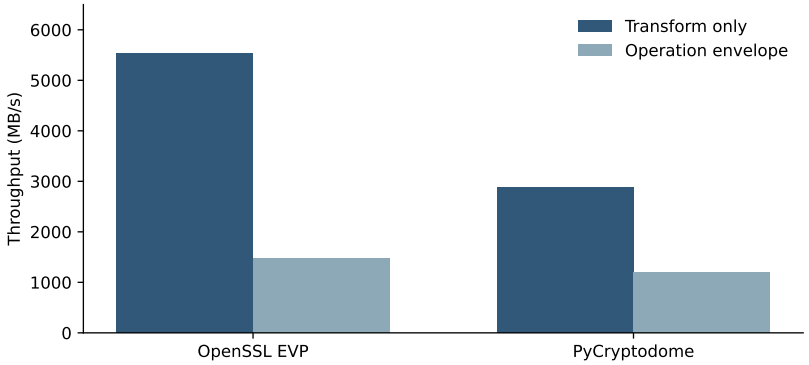


Fig. 2 — AES-128-CTR throughput at 32 MiB on Windows: transform processing and the instrumented operation. The broader interval includes preparation, result handling and benchmark diagnostics.

large improvement at every surrounding interface. Allocation, bindings, setup, result conversion and instrumentation contribute to the broader measurement, with different costs in each implementation. The measurements do not isolate their individual contributions, and subtracting aggregate medians would not provide a valid cost decomposition.

For application design, the relevant implication is to measure the interface actually used by the service. The instrumented operation demonstrates the sensitivity to measurement scope, but its diagnostic overhead prevents treating 1479 or 1202 MB/s as a prediction of production throughput. Buffer reuse, persistent contexts and different language bindings could change the relationship and require separate evaluation.

4.3 Key length and authenticated processing

The separate CBC experiment shows how key length affects both absolute throughput and relative acceleration (Table 2). Software encryption decreases from 225.1 MB/s for AES-128 to 174.9 MB/s for AES-256; the AES-NI path decreases from 531.6 to 474.0 MB/s. Nevertheless, the session-based acceleration factor increases from 2.37 to 2.71. Thus, a larger relative benefit from acceleration can coexist with lower absolute throughput. The pattern is consistent with the additional rounds required by longer keys, although the measured factors remain specific to this implementation and mode.

Authenticated processing introduces a different workload. Table 3 reports representative GCM sizes, and Fig. 3 includes all five evaluated sizes. At 1 KiB, AES-128 encryption reaches 70.6 MB/s, while verified decryption reaches 38.1

Table 2
AES-CBC encryption at 10 MiB on Windows (MB/s). Speedups are medians of session-level time ratios.

Key length	Software	AES-NI path	Speedup
AES-128	225.1	531.6	2.37×
AES-192	198.5	504.3	2.53×
AES-256	174.9	474.0	2.71×

MB/s. At 1 MiB, the corresponding rates rise to 1148.2 and 1131.4 MB/s. The encryption–decryption difference is therefore much more pronounced for short messages than for large buffers.

Table 3
AES-GCM payload throughput on Windows (MB/s). Encryption includes tag generation; decryption includes verification. Cipher-object construction is excluded.

Payload	Variant	Encrypt	Verify/decrypt
1 KiB	AES-128-GCM	70.6	38.1
1 KiB	AES-256-GCM	70.1	38.0
1 MiB	AES-128-GCM	1148.2	1131.4
1 MiB	AES-256-GCM	1059.3	1059.8
100 MiB	AES-128-GCM	1179.7	1189.3
100 MiB	AES-256-GCM	1100.4	1080.4

This pattern is consistent with fixed processing costs being distributed over more payload bytes as message size increases. The plateau at larger sizes suggests that additional payload then increases processing time approximately in proportion to its length over the measured range. AES-256 generally achieves lower throughput than AES-128, as expected from its additional rounds. However, the experiment does not separately time authentication multiplication, verification or allocation, so it cannot assign the observed size dependence to one component.

These GCM results complement the CTR and CBC observations by showing why a benchmark must represent the required cryptographic service. GCM includes authentication, whereas the CTR and CBC workloads reported here measure confidentiality operations. Their throughput figures therefore describe different functionality. The GCM timer also excludes object construction, so a service handling independent messages must additionally account for setup and its surrounding request-processing costs.

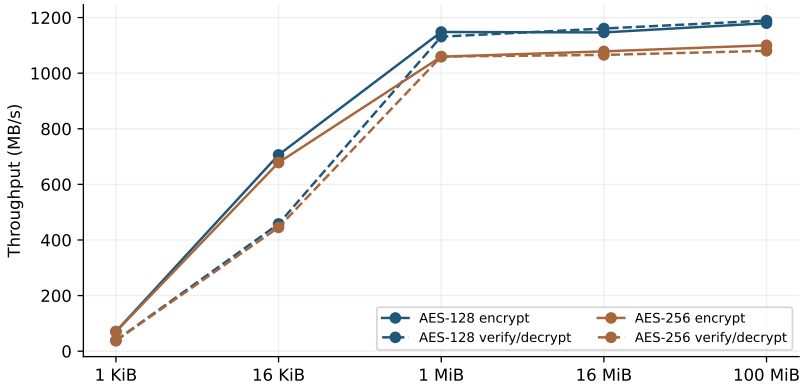


Fig. 3 — AES-GCM encryption and verified-decryption throughput on Windows for five payload sizes. Lines connect the measured aggregate rates.

5 General Discussion

5.1 Implications for encryption performance

The results identify three aspects of AES performance that should be considered together: the selected implementation, the work included in the timer and the cryptographic service being provided. The Arm64 measurements show a large difference between library configurations. The Windows measurements show that the relative advantage narrows at a broader software interface. The CBC and GCM experiments show that key length, authentication and message size further shape the processing cost. A useful performance statement must therefore specify these conditions rather than attach a single rate to AES or to a processor architecture.

This interpretation also clarifies the optimization question. When cipher processing dominates, a more efficient native implementation can offer substantial benefits. When other work occupies a larger part of the operation, optimizing that work becomes more important. The present data demonstrate the difference between measurement scopes, but do not determine the optimal balance for a particular application. That balance depends on its message distribution, buffer ownership, context lifetime and required security functionality.

5.2 Limitations and scope of generalization

The study contains repeated observations from one Windows laptop and one Arm64 cloud instance. Differences in processor configuration, operating system, OpenSSL version, scheduling and virtualization prevent a controlled architectural comparison. Processor affinity, dynamic frequency scaling and cloud contention

were not fully controlled. Repeated use of prepared buffers also differs from a cold-cache or continuous streaming workload. Five sessions describe repeatability under these conditions, not variability across a population of machines.

The measurement interfaces impose additional limits. PyCryptodome and OpenSSL transform timers include different language-interface costs, while the broader operation contains benchmark diagnostics. Capability inspection does not establish which instructions executed, and the Arm64 mask changes several capabilities simultaneously. The earlier CBC and GCM campaigns use their own timing conventions. These constraints permit comparisons within the stated scopes, but limit causal attribution and prevent combining all results into one acceleration factor.

Functional validation establishes agreement with expected outputs and, for GCM, rejection of the tested modifications. It does not establish resistance to timing leakage or other implementation attacks. Similarly, the study does not evaluate key management, nonce generation in a deployed service, energy consumption or latency under concurrent client load. Performance conclusions apply to the measured operations and should be considered alongside those separate requirements.

The transferable finding is methodological: a comparison should preserve the workload and identify the timed interval relevant to its intended use. Numerical rankings remain specific to the tested configurations, even when the same evaluation method is applied elsewhere.

5.3 Applications and analogies in other domains

The findings are relevant to CPU-based storage encryption, secure messaging and data-processing services. A service handling large buffers can evaluate whether native processing and reusable output storage reduce its total cost. A service handling short authenticated messages should assess setup and verification together with payload processing. These are hypotheses for application evaluation; the experiments do not demonstrate a speedup in a complete storage system or communication protocol.

A similar distinction arises in compression, numerical computing and machine-learning inference. A fast compression routine may be surrounded by buffer allocation and data copying; an optimized numerical kernel may require data conversion; an inference engine may spend substantial time on preprocessing and request handling. These examples illustrate the same question of measurement scope. They are conceptual analogies rather than additional empirical findings of this study.

A reproducible benchmark should report the mode, key size, payload dis-

tribution, nonce and AAD policy, library configuration, buffer policy and timer placement. Session-level variation should accompany aggregate rates. These details allow readers to assess whether a result represents the operation their application needs.

5.4 Future research

A first priority is a matched native benchmark on both architectures, with identical entry points and explicit alternatives for context and output-buffer reuse. Instruction tracing and independently controlled capability settings would then help separate AES processing from vector and authentication-related optimizations. Such experiments would provide stronger explanations for the observed library differences.

A second priority is authenticated encryption on Arm64 under the same nonce, AAD and verification policy used on Windows. Measuring object construction, payload processing and complete request latency separately would connect the present GCM observations to application behavior. Finally, evaluation within a storage or messaging service should include realistic message distributions, concurrency and input/output costs. This would test whether the implementation advantages observed here remain useful when encryption is one stage of a larger workflow.

6 Conclusion

This study examined how AES implementation choices and measurement scope affect CPU encryption throughput. The analyzed records show a strong dependence on the selected library path: at 32 MiB, default OpenSSL achieves approximately 4.03 GB/s on the measured Arm64 system, compared with 0.21–0.22 GB/s for the evaluated alternatives. On Windows, the OpenSSL-to-PyCryptodome throughput ratio decreases from approximately 1.93 for transformation to 1.23 for the instrumented operation. Separate CBC and GCM experiments show that key length and authenticated message processing introduce further workload-dependent effects.

Cipher throughput must therefore be interpreted for the software interface and cryptographic functionality relevant to the application. Implementation selection and surrounding software costs should be evaluated together. Although the numerical results are specific to the measured systems, the analysis provides a clearer basis for comparing encryption benchmarks. Matched native measurements and evaluation within storage or messaging services are needed to establish application-level benefits.

References

[1] National Institute of Standards and Technology. Advanced encryption standard (AES). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023. URL https://doi.org/10.6028/NIST.FIPS.197-upd1.

[2] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001. URL https://doi.org/10.6028/NIST.SP.800-38A.

[3] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007. URL https://doi.org/10.6028/NIST.SP.800-38D.

[4] Emilia Käsper and Peter Schwabe. Faster and timing-attack resistant AES-GCM. Cryptology ePrint Archive, Report 2009/129, International Association for Cryptologic Research, 2009. URL https://eprint.iacr.org/2009/129.

[5] Intel Corporation. Intel advanced encryption standard instructions (AES-NI), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[6] OpenSSL Project. EVP symmetric encryption and decryption. OpenSSL documentation, n.d. URL https://docs.openssl.org/3.0/man3/EVP_EncryptInit/.

[7] PyCryptodome. AES – advanced encryption standard. PyCryptodome documentation, n.d. URL https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html.

[8] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[9] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, pages 63–74, 2013. doi: 10.1145/2464157.2464160.